Capital Bikeshare Demand: Exploratory Analysis, Regression, and Forecasting

PhD Technical Assessment, Part 1, Research Summary Report

Data Source and Methodology

The UCI Bike Sharing Dataset (archive.ics.uci.edu/dataset/275) comes in two files. day.csv holds one row per calendar day, with 731 rows of daily totals. hour.csv holds one row per hour, with 17,379 rows. This analysis uses hour.csv throughout. The assessment asks for an hourly forecast of rental count, and day.csv has already collapsed each day into a single total, so it cannot support an hourly prediction task. Using it would have made Task 3 impossible from the start.

One thing worth flagging up front: the weather columns, temp, atemp, humidity, and windspeed, are pre-normalized by the dataset authors to a 0 to 1 scale, not given in their original units like degrees Celsius. This is noted here because it changes how regression coefficients are interpreted later on: a coefficient on temp describes the effect of moving across the whole observed temperature range, not the effect of a single degree.

Season and weathersit were treated as categorical variables rather than plain numbers throughout the regression work. Both are stored in the file as integer codes (season 1 through 4, for example), but those codes do not carry any real numeric ordering. Leaving them as numbers would force the model to assume winter is worth exactly twice as much as summer, which is not a meaningful assumption.

Task 1: Descriptive Statistics and Exploratory Analysis

The dataset covers two full years, January 2011 to December 2012, of hourly rentals in Washington, D.C. There are no missing values and no duplicate rows in the data that exists, but when a complete hourly timeline was rebuilt and compared to what was actually in the file, 165 hours were found to be missing entirely. This is addressed later, since it matters for how features are built for forecasting.

The target variable, cnt, is the total number of rentals in a given hour, and it is right-skewed: the mean is 189.5 but the median is only 142, which indicates a small number of very busy hours are pulling the average up. Figure 1 shows this directly. The left panel is the raw distribution, piled up near zero with a long tail stretching out to nearly 1,000 rentals in an hour. The right panel shows the same data after a log transform, which pulls the distribution back toward something closer to symmetric. This matters because it is an early sign that cnt behaves like count data rather than a normally distributed continuous variable, a point returned to in Task 2.

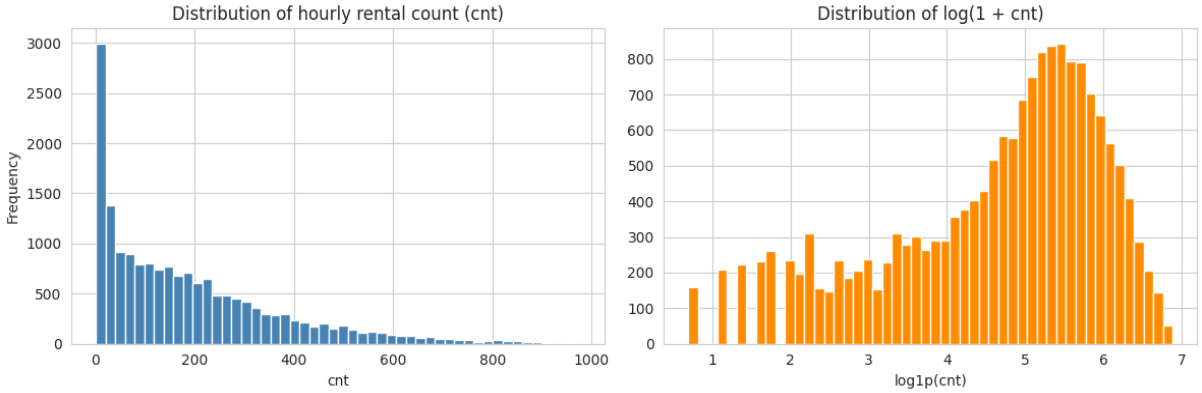


Figure 1: distribution of hourly rental count (left) and the same data after a log1p transform (right).

Figure 2 breaks demand down by hour of day, season, and working day status. The hourly pattern has two sharp peaks, one around 7 to 9 in the morning and another around 5 to 7 in the evening, with a dip overnight and a gentler bump through the middle of the day. That is a commuting signature, not a leisure one. Fall comes out as the highest average demand season, just ahead of summer, which likely reflects riders avoiding both the winter cold and the peak summer heat and humidity. Rentals are also somewhat higher on working days than on weekends in the aggregate.

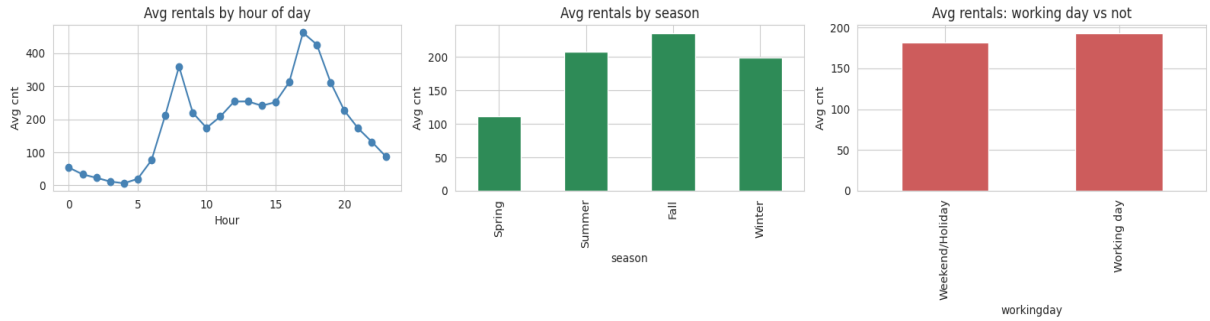


Figure 2: average rentals by hour of day, season, and working day status.

Splitting rentals by user type in Figure 3 explains a lot of what is happening in Figure 2. Registered members make up 81.2 percent of all rentals, and they ride far more on working days (167.6 per hour on average) than on weekends (124.0 per hour). Casual riders do the opposite: 57.4 per hour on weekends versus only 25.6 on working days. So the sharp double peak seen earlier is mostly a commuting pattern from registered members, while casual riders look more like weekend or leisure users. Two genuinely different groups of people are riding this system for different reasons, and the aggregate numbers are really a mix of both.

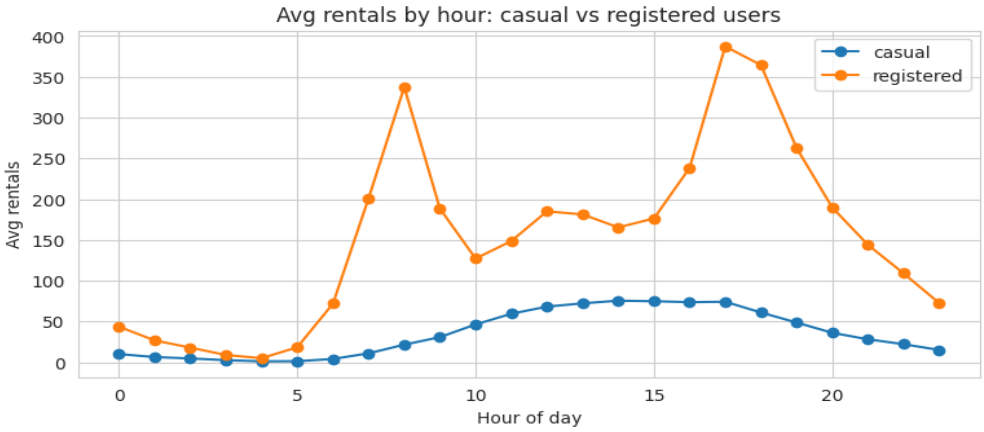


Figure 3: average hourly rentals, casual riders versus registered members.

Finally, Figure 4 looks at weather. The bar chart on the left shows a clean, orderly drop in average rentals as conditions worsen, from about 205 per hour when it is clear down to about 75 per hour in heavy rain or snow. The correlation matrix on the right shows something that needed to be known before running any regression: temp and atemp are correlated at 0.99, which makes sense since one is just a felt-temperature version of the other, but it also means putting both into a regression at once is going to cause problems. cnt itself correlates moderately with temp (0.40) and negatively with humidity (-0.32).

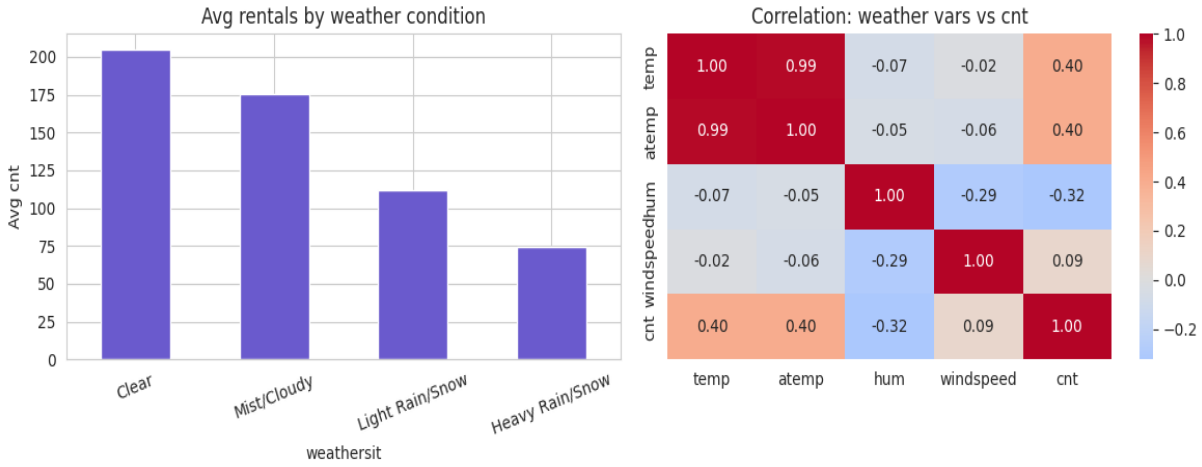


Figure 4: average rentals by weather condition (left) and correlation between weather variables and cnt (right).

Task 2: Statistical and Regression Analysis

An ordinary least squares regression of cnt on season, year, hour, holiday, weekday, working day status, weather category, temperature, humidity, and windspeed was fit first. It produced an R-squared of 0.399, and almost every predictor was statistically significant except fall season, working day status, and the rare heavy-rain weather category. Before trusting any of those coefficients, the model's assumptions were checked, and they did not hold up, in two specific ways.

First, multicollinearity. The Variance Inflation Factor for both temp and atemp came out around 43.7 and 43.8, far past the usual warning threshold of 10. That confirms what the correlation matrix in Figure 4 already hinted at, so atemp was dropped and temp was kept going forward.

Second, the residuals themselves were not well behaved. Figure 5 shows both diagnostics side by side. The left panel plots residuals against fitted values, and instead of a random cloud it forms a clear wedge shape, with the spread of errors growing as predicted demand grows. That is heteroscedasticity, meaning the model's errors are not constant across the range of predictions. The right panel is a standardized Q-Q plot, and the points curve away from the diagonal at the upper end rather than hugging it, showing the residuals are not normally distributed. Both problems trace back to the same root cause: cnt is skewed, non-negative count data, and OLS was never built with that kind of variable in mind.

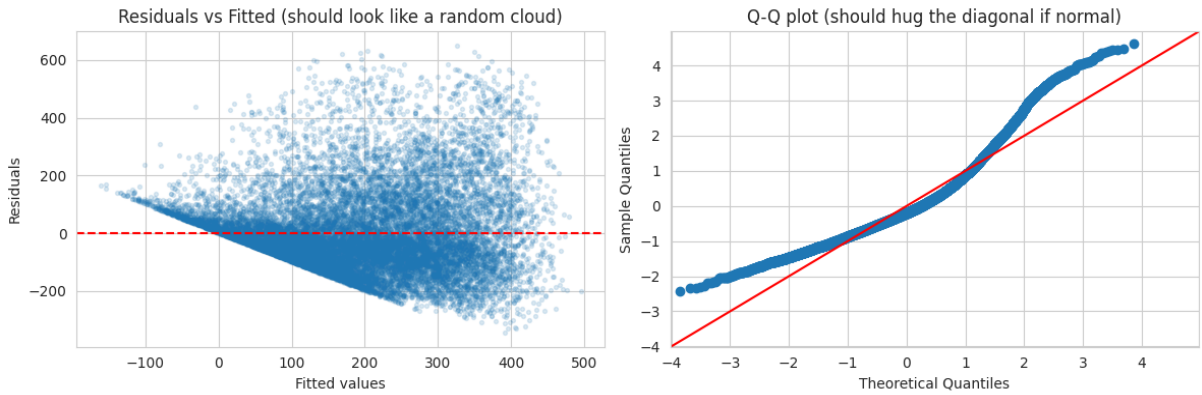


Figure 5: OLS residuals plotted against fitted values (left) and a standardized Q-Q plot of the same residuals (right).

To check how badly this matters, the variance of cnt was compared to its mean. If cnt behaved like a Poisson process, those two numbers would be roughly equal. Instead, the variance (32,901.5) is 173.7 times the mean (189.5). A plain Poisson regression was fit to confirm this formally, and its deviance to degrees of freedom ratio

came out at 94.7, when a value near 1 would indicate no overdispersion. That is about as clear a signal as could be asked for, so a Negative Binomial regression was used instead, which adds a dispersion parameter to soak up exactly this kind of extra variance. The fitted dispersion parameter came out at 0.80, confirming real overdispersion, and the Negative Binomial model's AIC of 208,429 is dramatically lower than the Poisson model's AIC of 1,755,740, meaning it fits the data far better.

Because a Negative Binomial model works on a log scale, its coefficients were exponentiated to read them as multiplicative effects on expected rentals rather than additive ones. The biggest effect by far was temperature: moving from the coldest to the warmest observed value is associated with a 538 percent increase in expected rentals, holding everything else fixed. Year 2 (2012) showed 60.7 percent higher expected rentals than year 1, which likely reflects the program simply growing in popularity over time rather than a weather or seasonal effect. Winter came out 58.7 percent higher than the spring baseline once temperature was already accounted for separately, a somewhat surprising result since Figure 2 showed fall as the highest raw season. This suggests that once temperature is controlled for directly, winter carries some additional independent effect on demand, perhaps drier air or less rain, that a simple bar chart cannot reveal on its own. Humidity carried the largest negative effect found: the highest observed humidity roughly halved expected rentals compared to the lowest. The heavy-rain weather category was not statistically significant, most likely because there are very few heavy-rain hours in two years of data to learn from.

Task 3: Demand Forecasting and Predictive Modeling

The assessment specifies an unusual partitioning rule: for every month, days 1 through 20 go into training and days 21 through the end of the month go into testing, applied across the whole two-year span rather than a single cutoff date. That means training and test rows are interleaved chronologically, which makes leakage prevention trickier than a normal forecast split.

The 165 missing hours flagged in Task 1 turned out to be a real problem here. If a feature like rentals one hour ago is built without accounting for gaps, a missing hour will silently shift that feature by more than one hour without any warning. To fix this properly, the timeline was first rebuilt so every hour exists as a row, with the missing ones showing up as explicit NaN values rather than absent rows. Short gaps of six hours or less were then forward-filled using only past values, never future ones, since using a future value to fill a gap would itself be a form of lookahead leakage. Gaps longer than six hours were left as missing and excluded rather than papered over. With that corrected series in place, lag features (1 hour, 24 hours, and 168 hours before the current hour) and rolling averages over the trailing 24 and 168 hours were built, all computed so that every feature for a given hour only uses information that had already happened in real time, regardless of whether the surrounding rows ended up in the training or test partition. Hour and month were also encoded using sine and cosine, so the model understands that hour 23 and hour 0 are one hour apart rather than at opposite ends of a number line.

For the model itself, an XGBoost regressor was trained on the lag, rolling, and cyclical features together with the weather and calendar variables from the earlier tasks. On the test set, it reached a mean absolute error of 22.9 rentals per hour, a root mean squared error of 36.8, and an R-squared of 0.96.

Model	MAE	RMSE	R-squared
Naive (same hour, last week)	59.1	101.9	0.69
Linear Regression	44.7	68.6	0.86
XGBoost	22.9	36.8	0.96

Table 1: forecasting performance of the naive baseline, Linear Regression, and XGBoost on the test set.

That R-squared of 0.96 was not taken at face value without checking whether it actually reflects good modeling, so XGBoost was compared against two baselines, shown in Table 1. The first is a naive forecast that just

guesses the same hour from one week earlier, and it already reaches an R-squared of 0.69 on its own, purely from how repetitive weekly ridership patterns are. The second is a plain Linear Regression trained on the exact same features as XGBoost, which improves to 0.86 by combining that weekly pattern with weather and calendar effects in a straight line. XGBoost nearly halves the error again, indicating the remaining gain is coming from non-linear interactions, for instance temperature mattering more at some hours than others, that a linear model simply cannot represent. Each step up in modeling complexity earned a real improvement, which is what makes XGBoost a justified choice here rather than an arbitrary one.

One honest limitation is worth flagging: about one percent of test rows still had a stray missing weather value after gap-filling. XGBoost tolerates missing values internally, so this stayed invisible until Linear Regression was fitted, which does not share that tolerance and raised an error. This was caught and those rows were explicitly dropped before fitting the baselines, rather than leaving it as a silent inconsistency.

Since the assessment specifically asks about performance across the evaluation windows, and not just one aggregate number, XGBoost's error was also broken down by each month's test window separately. Figure 6 shows this. The mean per-window MAE was 22.6 with a standard deviation of 5.4, ranging from a best window of 12.7 up to a worst window of 30.6. Looking at the trend, error is lowest early in 2011 and climbs gradually into 2012, which makes sense given that ridership itself grew substantially over that period. A fixed error of, say, 20 rentals represents a much smaller relative miss when typical demand is low than when it is high, so this pattern does not necessarily mean the model got worse over time in any relative sense.

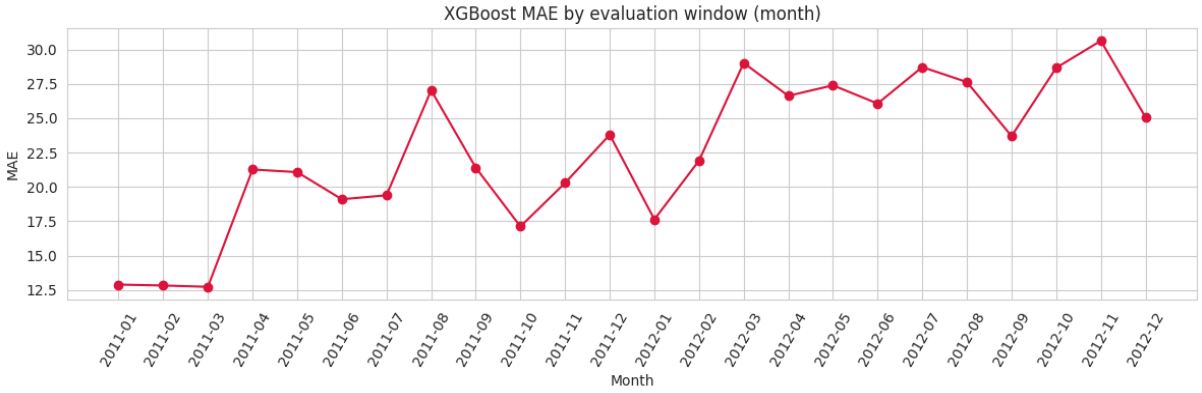


Figure 6: XGBoost mean absolute error for each month's evaluation window.

Figure 7 shows actual versus predicted rentals for representative stretches of the test set. The line is broken wherever there is a real gap between one month's test window and the next, rather than letting the plotting software draw a straight connecting line across weeks of missing test data, since that would be visually misleading. Where real data exists, the predicted line tracks the daily double peak closely and gets the timing of rises and falls right. It is a bit smoother than the real series, occasionally undershooting the very highest spikes and slightly overshooting smaller ones, which is typical, expected behavior for a tree-based model that tends to pull extreme values back toward the average rather than match them exactly.

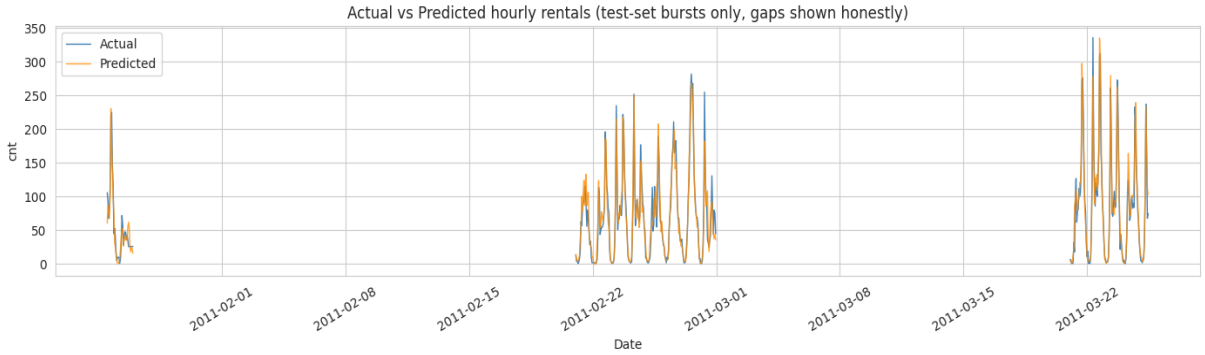


Figure 7: actual versus predicted hourly rentals across test-set windows.

Figure 8 shows which features XGBoost actually relied on. The three lag features together account for roughly 73 percent of total importance, with lag_168, meaning the same hour one week earlier, alone responsible for 42 percent. Weather variables barely register by comparison, with temp contributing under one percent. In plain terms, the model is mostly learning that ridership repeats itself week to week, and using weather as a secondary adjustment on top of that, rather than the other way around. This is worth stating plainly as a limitation: if an unusual weather event hit on a day that looked ordinary a week earlier, this model would likely be slow to react to it, since it is leaning so heavily on the prior week's pattern.

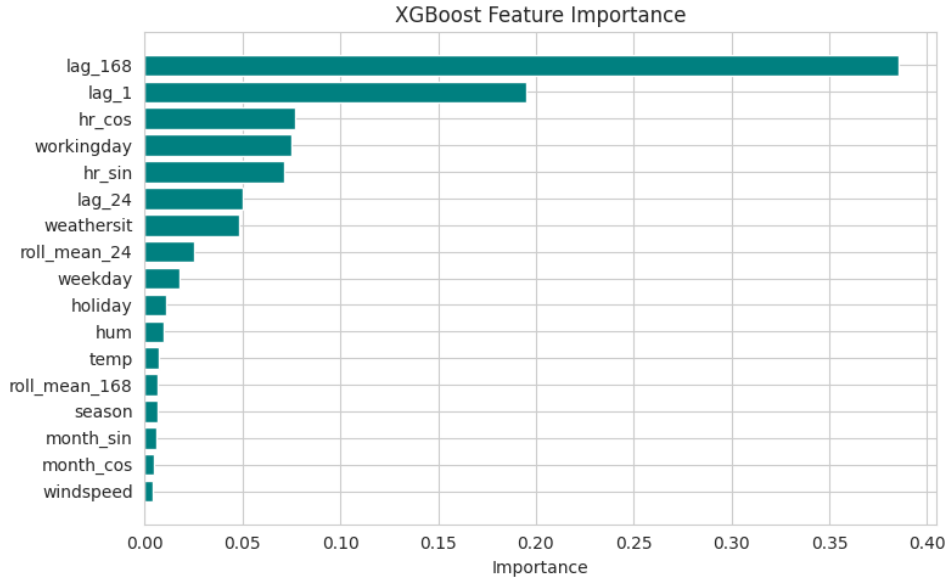


Figure 8: XGBoost feature importance.

Summary and Discussion

Stepping back from the details, here is what this whole assignment actually did and found, in plain terms. Two years of hour-by-hour bike rental records from Washington, D.C. were examined with three goals: describe the patterns in the data, explain what drives demand up or down, and build something that can predict future demand.

For the first part, the data told a fairly intuitive story once examined closely. This bike system is used in two different ways. A large group of registered members use it like a commute tool, riding heavily on weekday mornings and evenings and barely at all on weekends. A smaller group of casual, likely one-off or tourist riders use it the opposite way, mostly on weekends. Weather also matters a lot in an obvious direction: warm, dry, clear days bring out far more riders than cold, wet, or stormy ones.

For the second part, the goal was to go beyond just describing patterns and actually measure how big each effect is in a statistically defensible way. The tricky part was that rental counts are not the kind of nice, evenly-spread numbers that a standard regression expects. They are counts, they cannot go below zero, and they are lumpy, with lots of quiet hours and occasional very busy ones. A standard linear regression technically runs without crashing on data like this, but the assumptions behind it break down, as shown clearly by the residual plots. Switching to a model built for count data, called Negative Binomial regression, gave a much more honest way to quantify effects. That model showed temperature is by far the single biggest driver of demand, humidity has a large negative effect, and the bike-share program grew substantially in its second year of operation.

For the third part, prediction turned out to need a different toolkit than explanation did. The regression model was good at telling a clear causal-style story with interpretable numbers, but for pure forecasting accuracy, giving a model access to recent history, like what happened last week at this same hour, mattered far more than weather ever did. That history was built carefully so the model could never accidentally peek at the future, which took real care since the assignment's train and test periods are interleaved throughout the two years rather than neatly separated. The final model predicts hourly demand within about 23 rentals on average, which is a strong result, but this was not taken as a false sense of accuracy without scrutiny: it was compared against much simpler approaches and confirmed to meaningfully outperform them, and it is worth being upfront that most of its skill comes from repeating weekly patterns rather than truly understanding weather's effect on any given day.

Overall, the most important lesson from this assignment is that the right statistical tool depends on what the task actually is. Explaining why something happens and predicting what will happen next are genuinely different jobs, and they called for genuinely different models here, even though both were working with exactly the same underlying data.